Institute App: 4 Advanced Security Layers

Zanzibar (OpenFGA) + BeyondCorp Zero Trust + AWS Nitro Enclave + Post-Quantum Crypto — Native Android (Kotlin + Jetpack Compose) Integration Guide

13 November 2026

0. पहले ये पढ़ो (Read First)
1. चार layers एक नज़र में
2. Workflow Architecture
3. फ़ाइलें: क्या बनेगी, कहाँ बनेगी
4. Layer 1: Zanzibar-style Fine-Grained Authorization (OpenFGA)
5. Layer 2: BeyondCorp / Zero Trust
6. Layer 3: AWS Nitro Enclave (Confidential Computing, Data-in-Use)
7. Layer 4: Post-Quantum Cryptography (PQC)
8. GitHub Actions: सिर्फ़ नई workflow फ़ाइलें
9. Rollout Plan और Testing
10. आगे पढ़ने के लिए (Official references)

0. पहले ये पढ़ो (Read First)

ये गाइड आपके **Computer Institute Android App** (Kotlin + Jetpack Compose, native APK) में चार नई security layers जोड़ने का पूरा plan है: कौन सी फ़ाइलें बनेंगी, कहाँ बनेंगी, कैसे configure होंगी, कोड, और सब मिलकर कैसे काम करेंगी।

तीन सुनहरे नियम

1. **Additive only।** आपकी पुरानी libraries, security rules, functions, database, architecture और GitHub Actions workflow files **में कोई बदलाव नहीं।** सब कुछ *नई* फ़ाइलों/फ़ोल्डर्स में है। सिर्फ़ 3 जगह एक-एक लाइन जोड़नी पड़ती है (Section 3.3 में साफ़ लिखी है): यानी पुराना कोड बदलता नहीं, बस नई layer "plug-in" होती है।
2. **Enforcement हमेशा server पर।** Android app पर चलने वाला कोई भी check सिर्फ़ UI hint है (बटन दिखाना/छुपाना)। असली allow/deny गेटवे और enclave में होता है, क्योंकि फ़ोन पर बैठा attacker app को बदल सकता है।
3. **Phase-wise rollout।** चारों layers एक दिन में मत लगाओ। पहले shadow mode (सिर्फ़ log, block नहीं), फिर enforce। Section 9 में plan है।

माने गए Assumptions (अलग हों तो बता देना)

बात	जो मैंने माना
App	Native Android, Kotlin, Jetpack Compose, minSdk ~26
पुराना backend	कुछ भी हो सकता है (Firebase / REST / custom)। नई layers उसके **बगल में** बैठती हैं, उसे छूती नहीं
नई services	AWS Mumbai (`ap-south-1`) में
नया gateway की भाषा	Kotlin (Ktor), ताकि crypto कोड app और server में एक ही रहे। चाहें तो Node/Go में बदल सकते हैं

ईमानदार सीमाएँ (Honest limits) : (1) **AWS Nitro Enclave सिर्फ़ server-side है ।** Android फ़ोन पर Nitro नहीं चलता ; फ़ोन की सुरक्षा के लिए Keystore/StrongBox + Play Integrity है । (2) **Zanzibar खुद कोई downloadable software नहीं है**, ये Google का internal system है । हम इसकी *design* (relation tuples) का open-source रूप **OpenFGA** use करेंगे । (3) अगर आपका app **सीधे Firebase/Firestore** से बात करता है, तो वो calls नए gateway से होकर नहीं गुज़रतीं ; उन पर आपके *पुराने* security rules ही लागू रहेंगे (unchanged) । संवेदनशील operations को धीरे-धीरे gateway के रास्ते पर लाना है । (4) नीचे के कोड **starter code** हैं । Library versions (BouncyCastle, OkHttp, Play Integrity) अपने project के हिसाब से latest stable से मिलाना और unit tests चलाना ज़रूरी है । इस sandbox में मैं इन्हें compile नहीं कर पाया ।

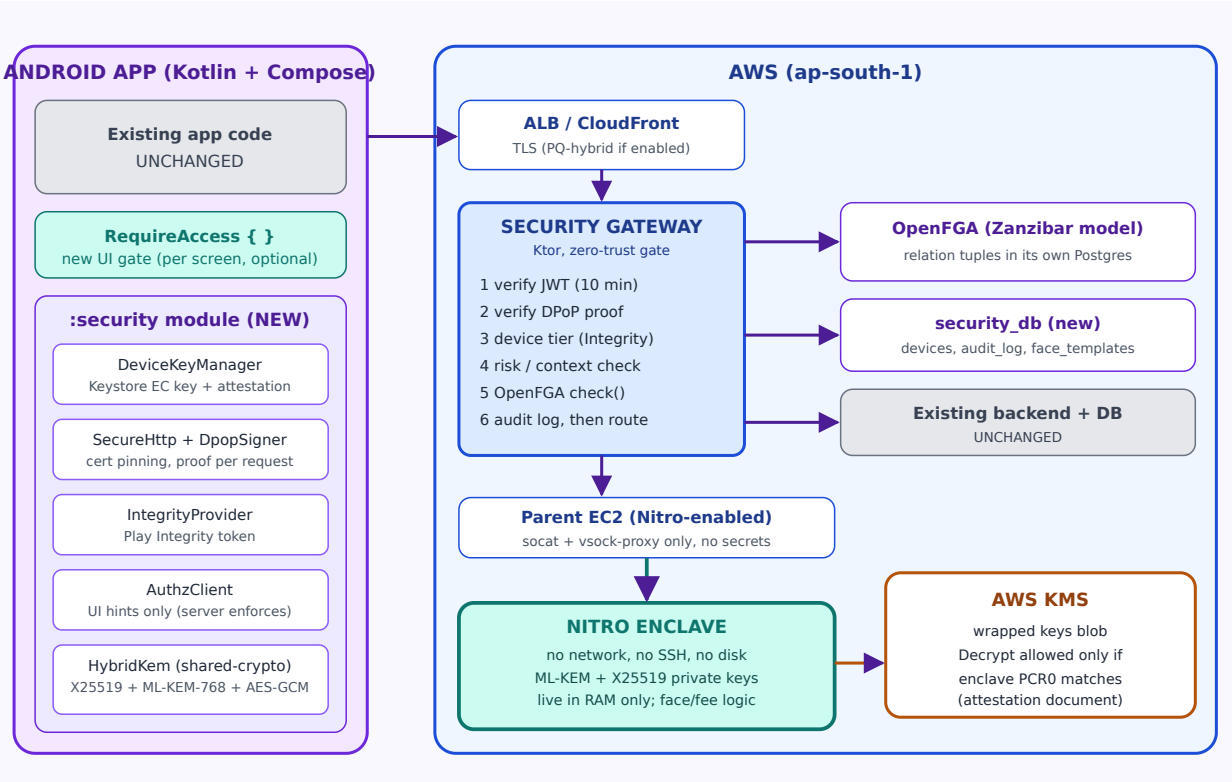
1. चार layers एक नज़र में

#	Layer	क्या बचाती है	कहाँ चलती है	Institute में उदाहरण
1	**Zanzibar-style fine-grained authz** (OpenFGA)	"कौन, किस चीज़ पर, क्या कर सकता है ?" हर object पर relation-based permission	Server (gateway + OpenFGA)	Teacher सिर्फ़ *अपने batch* की attendance लगाए ; student सिर्फ़ *अपनी* fee देखे ; parent सिर्फ़ अपने बच्चे की
2	**BeyondCorp / Zero Trust**	"Network या सिर्फ़ login पर भरोसा नहीं ।" हर request पर user + device + context जाँचो	Android (device key, Play Integrity) + gateway	Rooted/emulator फ़ोन से admin portal नहीं खुलेगा ; चोरी हुए token का दूसरे फ़ोन पर use बेकार
3	**AWS Nitro Enclave** (Confidential Computing)	Data **use होते समय** भी encrypted-isolated: server का admin/root भी नहीं देख सकता	Server (EC2 + enclave)	Face embeddings और fee data enclave के अंदर ही process हों
4	**Post-Quantum Crypto (PQC)**	"Harvest now, decrypt later" से बचाव : आज का डेटा कल के quantum computer से न खुले	Android + gateway + enclave	Biometric/fee payload पर hybrid X25519 + **ML-KEM-768** (NIST FIPS 203) encryption

आपस का रिश्ता : Layer 2 बताती है "ये request भरोसेमंद फ़ोन और सही व्यक्ति से आई" ; Layer 1 बताती है "इस व्यक्ति को इस object पर ये action करने की इजाज़त है" ; Layer 4 payload को आगे के लिए सुरक्षित लॉक करती है ; Layer 3 उस लॉक की चाबी सिर्फ़ verified enclave को देती है ।

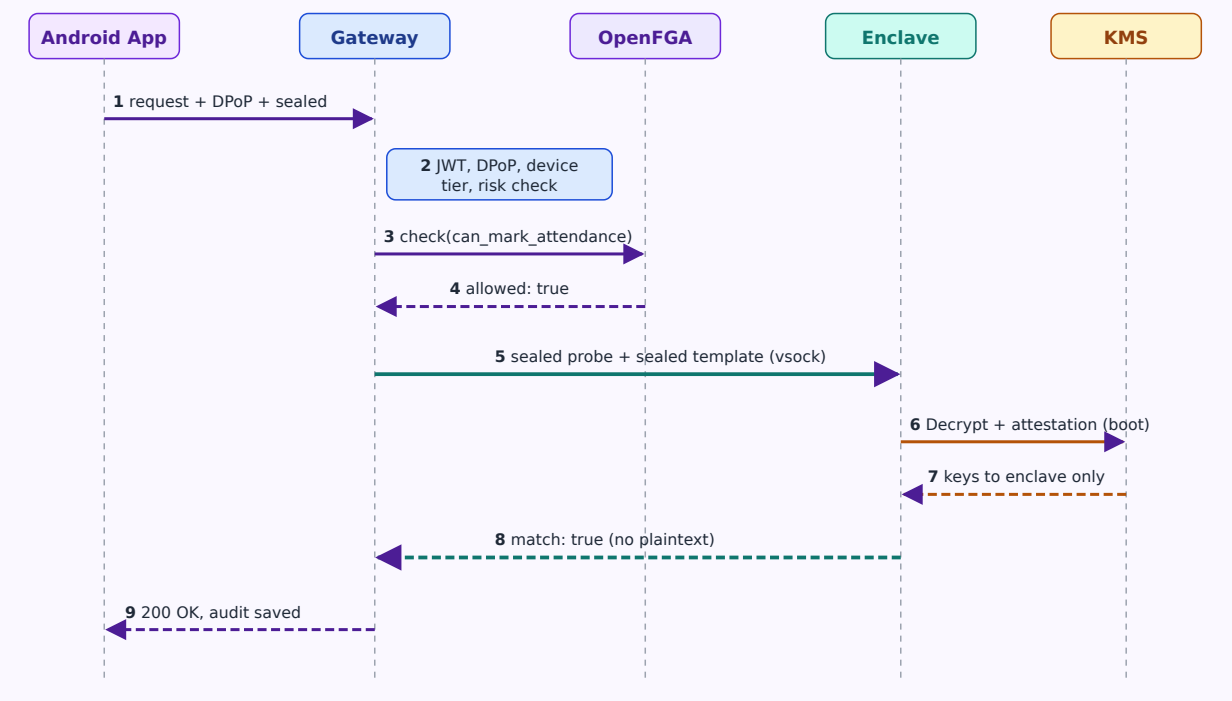
2. Workflow Architecture

2.1 Component diagram



Components: Android app, Security Gateway, OpenFGA, Nitro Enclave, KMS

2.2 एक request का पूरा सफ़र (Face Attendance उदाहरण)



Request lifecycle sequence

Step	क्या होता है	कौन करता है
1	App एक request बनाती है : JWT + **DPoP proof** (Keystore key से signed) + payload जो **hybrid PQC से sealed** है	`:security` module
2	Gateway: JWT valid? DPoP proof उसी device key से ? device revoked तो नहीं ? Play Integrity से device tier काफ़ी है ? risk signals ठीक ?	Gateway (Layer 2)
3-4	Gateway OpenFGA से पूछता है : `user:teacher_01` क्या `batch:cpp2026` पर `can_mark_attendance` है ?	Gateway + OpenFGA (Layer 1)
5	Allowed तो sealed payload enclave को जाता है (parent EC2 सिर्फ़ vsock bridge है, plaintext नहीं देख सकता)	Gateway → Enclave (Layer 3)
6-7	**Boot के समय** enclave अपने attestation document से KMS को साबित करता है कि वो वही approved image है, तब KMS उसे private keys का blob खोलकर देता है	Enclave + KMS
8	Enclave के अंदर payload खुलता है (Layer 4 की ML-KEM + X25519 चाबी से), face match होता है, सिर्फ़ `match: true/false` बाहर आता है	Enclave
9	Gateway response लौटाता है और audit log में लिखता है	Gateway

3. फ़ाइलें: क्या बनेगी, कहाँ बनेगी

3.1 नया folder structure (repo root में)

```
<your-repo>/
├── security/                          # NEW  Android library module  (:security)
│   ├── build.gradle.kts
│   ├── security.properties            # public config (URL, pins, public keys). No secrets
│   └── src/main/
│       ├── AndroidManifest.xml        # androidx.startup auto-init
│       └── kotlin/com/institute/security/
│           ├── SecurityInitializer.kt
│           ├── SecurityKit.kt
│           ├── SecurityConfig.kt
│           ├── authz/AuthzClient.kt           # Layer 1 (UI hints)
│           ├── device/DeviceKeyManager.kt     # Layer 2
│           ├── device/DpopSigner.kt           # Layer 2
│           ├── device/IntegrityProvider.kt    # Layer 2
│           ├── net/SecureHttp.kt              # Layer 2 (pinning + interceptor)
│           ├── net/SessionManager.kt          # Layer 2
│           └── pqc/SecureEnvelope.kt          # Layer 4 (uses shared-crypto)
│
├── shared-crypto/                     # NEW  pure-Kotlin source folder (no module)
│   └── src/main/kotlin/com/institute/crypto/HybridKem.kt   # Layer 4, shared by app+gateway+enclave
│
├── app/src/main/java/<your.pkg>/ui/security/
│   └── RequireAccess.kt               # NEW  Compose gate (file added inside your app module)
│
├── security-gateway/                  # NEW  Ktor service (its own Gradle build)
│   ├── settings.gradle.kts
```

```
│   ├── build.gradle.kts
│   ├── src/main/kotlin/com/institute/gateway/
│   │   ├── Main.kt   Gate.kt   Policy.kt   FgaClient.kt   Enroll.kt   IntegrityVerifier.kt   EnclaveClient.kt
│   ├── src/main/resources/db/001_security_db.sql
│   └── tools/KeyTool.kt              # one-time PQC key generation
│
├── authz/                            # NEW  Layer 1 definitions
│   ├── model.fga
│   ├── model.fga.yaml                # tests
│   └── seed-tuples.sh
│
├── enclave/                          # NEW  Layer 3
│   ├── Dockerfile
│   ├── run.sh
│   ├── vendor/kmstool_enclave_cli    # built from aws-nitro-enclaves-sdk-c
│   └── src/main/kotlin/com/institute/enclave/EnclaveMain.kt
│
├── infra/                            # NEW  AWS setup
│   ├── kms-key-policy.json
│   ├── allocator.yaml
│   ├── vsock-proxy.yaml
│   └── parent-bootstrap.sh
│
└── .github/workflows/                # NEW files only (old workflows untouched)
    ├── sec-authz-model-test.yml
    ├── sec-gateway-ci.yml
    ├── sec-android-module.yml
    └── sec-enclave-build.yml
```

3.2 ये सब UNCHANGED रहेगा

चीज़	स्थिति
पुरानी Gradle libraries / version catalog (`libs.versions.toml`)	**नहीं बदलती** (नया module अपनी dependencies अपने `build.gradle.kts` में रखता है)
पुराने security rules (Firestore/DB rules, `network_security_config.xml` , ProGuard)	**नहीं बदलते**: pinning हम OkHttp `CertificatePinner` से अपने client में करते हैं
पुराने functions / backend code / database / schema	**नहीं बदलते**: नया `security_db` अलग है
App architecture (MVVM/Hilt/Navigation आदि)	**नहीं बदलता**
पुराने GitHub Actions workflow files	**नहीं बदलते**: सिर्फ़ `sec-*.yml` नई फ़ाइलें जुड़ती हैं

3.3 सिर्फ़ 3 “plug” लाइनें (additive, कुछ हटता नहीं)

```
        // (a) settings.gradle.kts  : module register
        include(":security")

        // (b) app/build.gradle.kts  : dependencies { ... }  के अंदर
        implementation(project(":security"))

        // (c) Application.onCreate() या जहाँ आपका login token मिलता है:
        //     "मेरा मौजूदा login token ये है" बताने वाली एक लाइन
        SecurityKit.identityProvider = { myExistingAuth.currentIdToken() }
```

इसके अलावा किसी screen को protect करना हो तो उसे `RequireAccess { ... }` में लपेटना **आपकी मर्ज़ी** (opt-in) है, पुरानी screens अपने-आप नहीं बदलतीं।

> **Mode A (shadow) बनाम Mode B (enforce):** शुरू में गेटवे नए endpoints के लिए *बगल में* चलता है। जब आप तैयार हों, तब पुराने sensitive calls का base URL गेटवे की तरफ़ मोड़ें (एक config लाइन)। तब तक पुराना सिस्टम बिल्कुल पहले जैसा चलता है।

4. Layer 1: Zanzibar-style Fine-Grained Authorization (OpenFGA)

4.1 Zanzibar का विचार (2 मिनट में)

Google के Zanzibar paper का मूल विचार: हर permission एक **relation tuple** है: `object # relation @ user`। जैसे `batch:cpp2026#teacher@user:teacher_01`। फिर **model** बताता है कि relations एक दूसरे से कैसे बनती हैं (जैसे "institute का admin, उसके हर batch का भी admin है")। check करते समय tuples का graph चलकर जवाब निकलता है। इससे "Role-based" की जगह **object-level** (ReBAC) control मिलता है।

हम **OpenFGA** (open-source, Zanzibar से प्रेरित, CNCF project) use करेंगे। विकल्प: SpiceDB, Ory Keto। Model तीनों में लगभग एक जैसा सोचा जाता है।

4.2 OpenFGA चलाना (Mumbai में Docker/ECS पर)

```
        # local test के लिए
        docker run -d --name openfga -p 8080:8080 -p 3000:3000 openfga/openfga run

        # production: Postgres के साथ (अलग DB, पुराने DB से अलग)
        docker run -d --name openfga -p 8080:8080 \
          -e OPENFGA_DATASTORE_ENGINE=postgres \
          -e OPENFGA_DATASTORE_URI='postgres://fga:***@fga-db:5432/fga?sslmode=require' \
          -e OPENFGA_AUTHN_METHOD=preshared \
          -e OPENFGA_AUTHN_PRESHARED_KEYS='<LONG_RANDOM_KEY>' \
          openfga/openfga run
        # पहली बार migration:  openfga migrate --datastore-engine postgres --datastore-uri '...'
```

ज़रूरी: OpenFGA को **इंटरनेट पर expose मत करो**। सिर्फ़ गेटवे (private subnet) इससे बात करे। App कभी सीधे OpenFGA को call नहीं करेगा।

4.3 `authz/model.fga`: Institute का authorization model

```
model
  schema 1.1

type user

type institute
  relations
    define admin: [user]
    define staff: [user]

type course
  relations
    define institute: [institute]
    define instructor: [user]
    define enrolled: [user]
    define can_view_content: instructor or enrolled or admin from institute
    define can_edit_content: instructor or admin from institute

type batch
  relations
    define institute: [institute]
    define course: [course]
    define teacher: [user]
    define student: [user]
    define can_mark_attendance: teacher or admin from institute
    define can_view_roster: teacher or student or admin from institute

type student_profile
  relations
```

```
    define institute: [institute]
    define batch: [batch]
    define owner: [user]
    define guardian: [user]
    define can_view: owner or guardian or teacher from batch or admin from institute
    define can_edit: admin from institute

type fee_record
  relations
    define institute: [institute]
    define profile: [student_profile]
    define can_view: owner from profile or guardian from profile or admin from institute
    define can_edit: admin from institute
```

4.4 Store बनाना + model upload + tuples लिखना

```
        # FGA CLI (docker से)
        alias fga='docker run --rm --network host -v $PWD/authz:/w -w /w openfga/cli'

        fga store create --name institute            # -> store id नोट करो
        export FGA_STORE_ID=<store_id>
        fga model write --store-id=$FGA_STORE_ID --file model.fga   # -> authorization_model_id नोट करो
```

`authz/seed-tuples.sh` : **अपने** डेटा से tuples बनाने का उदाहरण (असल में ये gateway के admin endpoints से auto-write होंगे, जैसे नया admission = नए tuples) :

```
        # Institute और admin
        fga tuple write --store-id=$FGA_STORE_ID user:anita admin institute:main
        # Batch, teacher, student
        fga tuple write --store-id=$FGA_STORE_ID institute:main institute batch:cpp2026
        fga tuple write --store-id=$FGA_STORE_ID user:teacher_01 teacher batch:cpp2026
        fga tuple write --store-id=$FGA_STORE_ID user:rahul student batch:cpp2026
        # Student profile + fee
        fga tuple write --store-id=$FGA_STORE_ID institute:main institute student_profile:rahul
        fga tuple write --store-id=$FGA_STORE_ID batch:cpp2026 batch student_profile:rahul
        fga tuple write --store-id=$FGA_STORE_ID user:rahul owner student_profile:rahul
        fga tuple write --store-id=$FGA_STORE_ID user:rahul_dad guardian student_profile:rahul
        fga tuple write --store-id=$FGA_STORE_ID institute:main institute fee_record:rahul_nov
        fga tuple write --store-id=$FGA_STORE_ID student_profile:rahul profile fee_record:rahul_nov

        # जाँच
        fga query check --store-id=$FGA_STORE_ID user:teacher_01 can_mark_attendance batch:cpp2026   #
allowed: true
        fga query check --store-id=$FGA_STORE_ID user:teacher_01 can_view fee_record:rahul_nov       #
allowed: false
```

4.5 Model के tests (`authz/model.fga.yaml`), CI में चलेंगे

```
        name: institute authz tests
        model_file: model.fga
        tuples:
          - {user: user:anita, relation: admin, object: institute:main}
          - {user: institute:main, relation: institute, object: batch:cpp2026}
          - {user: user:teacher_01, relation: teacher, object: batch:cpp2026}
          - {user: institute:main, relation: institute, object: student_profile:rahul}
          - {user: batch:cpp2026, relation: batch, object: student_profile:rahul}
          - {user: user:rahul, relation: owner, object: student_profile:rahul}
          - {user: institute:main, relation: institute, object: fee_record:rahul_nov}
          - {user: student_profile:rahul, relation: profile, object: fee_record:rahul_nov}
        tests:
          - name: teacher marks own batch only
            check:
              - user: user:teacher_01
                object: batch:cpp2026
                assertions: {can_mark_attendance: true}
          - name: fee privacy
            check:
```

```
  - user: user:rahul
    object: fee_record:rahul_nov
    assertions: {can_view: true, can_edit: false}
  - user: user:teacher_01
    object: fee_record:rahul_nov
    assertions: {can_view: false}
  - user: user:anita
    object: fee_record:rahul_nov
    assertions: {can_view: true, can_edit: true}
```

Run: `docker run --rm -v $PWD/authz:/w -w /w openfga/cli model test --tests model.fga.yaml`

4.6 Gateway side: `FgaClient.kt`

```
package com.institute.gateway

import io.ktor.client.*
import io.ktor.client.call.*
import io.ktor.client.request.*
import io.ktor.http.*
import kotlinx.serialization.Serializable

@Serializable data class TupleKey(val user: String, val relation: String, val `object`: String)
@Serializable data class CheckReq(val tuple_key: TupleKey, val authorization_model_id: String)
@Serializable data class CheckRes(val allowed: Boolean = false)

class FgaClient(
    private val http: HttpClient, private val base: String,
    private val storeId: String, private val modelId: String, private val apiKey: String,
) {
    suspend fun check(user: String, relation: String, obj: String): Boolean = try {
        http.post("$base/stores/$storeId/check") {
            bearerAuth(apiKey)
            contentType(ContentType.Application.Json)
            setBody(CheckReq(TupleKey(user, relation, obj), modelId))
        }.body<CheckRes>().allowed
    } catch (e: Exception) { false }        // FAIL CLOSED: error = deny
}
```

4.7 Android side: UI hint + Compose gate

`security/.../authz/AuthzClient.kt`:

```
class AuthzClient(private val cfg: SecurityConfig, private val client: OkHttpClient) {
    private data class Entry(val allowed: Boolean, val at: Long)
    private val cache = java.util.concurrent.ConcurrentHashMap<String, Entry>()

    /** सिर्फ़ UI hint. असली enforcement server पर है. 60 सेकंड cache. */
    suspend fun can(relation: String, obj: String): Boolean = withContext(Dispatchers.IO) {
        val key = "$relation|$obj"
        cache[key]?.takeIf { System.currentTimeMillis() - it.at < 60_000 }?.let { return@withContext
it.allowed }
        val json = org.json.JSONObject().put("relation", relation).put("object", obj).toString()
        val req = Request.Builder().url("${cfg.gatewayBaseUrl}/v1/authz/check")
            .post(json.toRequestBody("application/json".toMediaType())).build()
        val allowed = try {
            client.newCall(req).execute().use { r ->
                r.isSuccessful && org.json.JSONObject(r.body!!.string()).optBoolean("allowed", false)
            }
        } catch (e: Exception) { false }
        cache[key] = Entry(allowed, System.currentTimeMillis()); allowed
    }
    fun clear() = cache.clear()
}
```

`app/.../ui/security/RequireAccess.kt` (आपके app module में **नई फ़ाइल**):

```
@Composable
fun RequireAccess(
    relation: String, obj: String,
    denied: @Composable () -> Unit = {},
    content: @Composable () -> Unit,
) {
    var allowed by remember(relation, obj) { mutableStateOf<Boolean?>(null) }
    LaunchedEffect(relation, obj) { allowed = SecurityKit.authz.can(relation, obj) }
    when (allowed) { true -> content(); false -> denied(); null -> Unit }
}

// इस्तेमाल (जहाँ चाहें):
// RequireAccess("can_mark_attendance", "batch:$batchId") { MarkAttendanceButton() }
```

5. Layer 2: BeyondCorp / Zero Trust

5.1 सिद्धांत (BeyondCorp से)

- "Office network = भरोसेमंद" वाला विचार छोड़ो। **हर request** का फ़ैसला तीन चीज़ों से: **User** (कौन), **Device** (किस फ़ोन से, कितना भरोसेमंद), **Context** (कब, कहाँ से, असामान्य तो नहीं)।
- Token छोटा-जीवन वाला (10 मिनट) और **device की चाबी से बँधा** (DPoP, RFC 9449), ताकि token चोरी होकर दूसरे फ़ोन पर बेकार हो।
- हर action की अपनी **न्यूनतम device-tier** (नीचे table)।

5.2 Device tiers और policy table

Play Integrity के verdict से tier बनता है। **सावधानी:** सब जगह STRONG माँगोगे तो कई सस्ते फ़ोन बाहर हो जाएँगे, इसलिए tier action के जोखिम के हिसाब से रखो।

Action	Min tier	अतिरिक्त शर्त
Course notes देखना	`BASIC`	valid session
Fee देखना	`DEVICE`	OpenFGA `can_view`
Teacher attendance (face)	`DEVICE`	hardware-backed key attestation + liveness + `can_mark_attendance`
Admin portal / fee edit	`STRONG`	integrity token < 5 min पुराना + biometric prompt

5.3 Gateway: SQL (`db/001_security_db.sql`, नया अलग database)

```
CREATE TABLE devices (
  id               UUID PRIMARY KEY,
  user_id          TEXT NOT NULL,
  pubkey_jkt       TEXT NOT NULL UNIQUE,   -- JWK thumbprint (DPoP binding)
  pubkey_jwk       JSONB NOT NULL,
  attestation_lvl  TEXT NOT NULL,          -- SOFTWARE | TRUSTED_ENVIRONMENT | STRONGBOX
  os_patch_level   TEXT,
  enrolled_at      TIMESTAMPTZ DEFAULT now(),
  last_seen        TIMESTAMPTZ,
  revoked_at       TIMESTAMPTZ             -- खोया फ़ोन: admin यहाँ set करे
);
CREATE TABLE audit_log (
  id BIGSERIAL PRIMARY KEY, ts TIMESTAMPTZ DEFAULT now(),
  user_id TEXT, device_id UUID, action TEXT, object TEXT,
  decision TEXT, tier TEXT, ip INET, detail JSONB
);
CREATE TABLE face_templates (            -- Layer 3/4: templates sealed हैं, plaintext नहीं
```

```
          user_id TEXT PRIMARY KEY, kid TEXT NOT NULL, sealed JSONB NOT NULL, updated_at TIMESTAMPTZ DEFAULT
now()
        );
        CREATE TABLE replay_nonces (nonce TEXT PRIMARY KEY, seen_at TIMESTAMPTZ DEFAULT now());  -- 10 min बाद
delete job
```

5.4 Gateway: `Gate.kt` (पूरे Zero-Trust pipeline का दिल)

```
        enum class Tier { UNTRUSTED, BASIC, DEVICE, STRONG }

        class Policy(
            val minTier: Tier,
            val relation: String? = null,                          // OpenFGA relation
            val objectOf: ((ApplicationCall) -> String)? = null,  // जैसे "batch:${id}"
            val maxIntegrityAgeSec: Long = 3600,
        )

        class Denied(val status: Int, msg: String) : RuntimeException(msg)
        data class Principal(val userId: String, val deviceId: String, val tier: Tier)

        suspend fun ApplicationCall.gate(d: Deps, p: Policy): Principal {
            val auth = request.header("Authorization")?.removePrefix("DPoP ") ?: throw Denied(401, "no
token")
            val claims = d.jwt.verify(auth)                                    // 1) identity, exp <= 10 min
            d.dpop.verify(                                                     // 2) proof-of-possession (RFC
9449)
                proof = request.header("DPoP") ?: throw Denied(401, "no proof"),
                method = request.httpMethod.value, url = d.publicUrl(this), accessToken = auth, expectedJkt =
claims.cnfJkt)
            val device = d.devices.find(claims.deviceId) ?: throw Denied(401, "unknown device")
            if (device.revokedAt != null) throw Denied(403, "device revoked")
            val tier = d.integrity.tierFor(device, request.header("X-Integrity"), p.maxIntegrityAgeSec)  //
3) posture
            if (tier < p.minTier) throw Denied(403, "device tier too low")
            d.risk.assess(claims.sub, device, request.origin.remoteHost)       // 4) context (impossible
travel, odd hour...)
            if (p.relation != null) {                                          // 5) Zanzibar/OpenFGA
                val obj = p.objectOf!!(this)
                if (!d.fga.check("user:${claims.sub}", p.relation, obj)) {
                    d.audit.log(claims.sub, device.id, request.uri, obj, "DENY", tier); throw Denied(403, "no
permission")
                }
            }
            d.audit.log(claims.sub, device.id, request.uri, p.relation, "ALLOW", tier)   // 6) audit
            return Principal(claims.sub, device.id, tier)
        }

        // एक route:
        post("/v1/attendance/{batchId}/face") {
            val me = call.gate(deps, Policy(Tier.DEVICE, "can_mark_attendance") {
"batch:${it.parameters["batchId"]}" })
            val body = call.receive<FaceVerifyRequest>()
            call.respond(deps.enclave.verifyFace(body, me))     // सिर्फ़ result बाहर आता है
        }
```

Shadow mode: `Deps` में `ENFORCE=false` flag रखो। तब `Denied` की जगह सिर्फ़ audit में `WOULD_DENY` लिखो। 1-2 हफ़्ते log देखकर policies ठीक करो, फिर `ENFORCE=true`।

5.5 Gateway: Play Integrity और Key Attestation (संक्षेप में)

- **Play Integrity verdict decode:** Google Play Integrity API का `decodeIntegrityToken` (Google service account से, server-side)। जाँचो: `requestDetails.requestHash` == आपकी action की hash, `timestampMillis` ताज़ा, `appIntegrity.appRecognitionVerdict == PLAY_RECOGNIZED`, और `deviceIntegrity.deviceRecognitionVerdict` से tier: `MEETS_BASIC_INTEGRITY` → BASIC, `MEETS_DEVICE_INTEGRITY` → DEVICE, `MEETS_STRONG_INTEGRITY` → STRONG।

- **Key attestation (enrollment के समय एक बार):** app Keystore key का certificate chain भेजती है। Server (Google की open-source `android-key-attestation` library से) जाँचता है: chain Google की root तक जाती है, `attestationChallenge` आपकी दी हुई nonce है, `attestationSecurityLevel` ≥ TrustedEnvironment, और key का public hash वही है जो DPoP में आएगा। तब `devices` में row बनती है।

5.6 Android: `:security` module

`security/build.gradle.kts`

```
        plugins {
            id("com.android.library")
            id("org.jetbrains.kotlin.android")
            id("org.jetbrains.kotlin.plugin.serialization")
        }   // plugin IDs वही रखो जो app/build.gradle.kts में हैं

        val sec = java.util.Properties().apply {
            file("security.properties").takeIf { it.exists() }?.inputStream()?.use(::load)
        }

        android {
            namespace = "com.institute.security"
            compileSdk = 35                       // app जैसा
            defaultConfig {
                minSdk = 26                       // app जैसा
                buildConfigField("String", "GATEWAY_URL", "\"${sec["gateway.url"]}\"")
                buildConfigField("String", "TLS_PINS", "\"${sec["tls.pins"]}\"")          //
"sha256/AAA=,sha256/BBB="
                buildConfigField("String", "PLAY_CLOUD_PROJECT", "\"${sec["play.cloudProject"]}\"")
                buildConfigField("String", "PQC_KID", "\"${sec["pqc.kid"]}\"")
                buildConfigField("String", "PQC_MLKEM_PUB", "\"${sec["pqc.mlkem.pub"]}\"")
                buildConfigField("String", "PQC_X25519_PUB", "\"${sec["pqc.x25519.pub"]}\"")
            }
            buildFeatures { buildConfig = true }
            sourceSets["main"].java.srcDir("../shared-crypto/src/main/kotlin")   // shared crypto
            compileOptions { sourceCompatibility = JavaVersion.VERSION_17; targetCompatibility =
JavaVersion.VERSION_17 }
            kotlinOptions { jvmTarget = "17" }
        }

        dependencies {
            // NOTE: OkHttp / coroutines / serialization के version वही रखो जो आपके app में हैं
            // (वरना Gradle आपकी पुरानी library को upgrade कर देगा, जो हम नहीं चाहते)
            implementation("org.bouncycastle:bcprov-jdk18on:1.80")          // ML-KEM, X25519 (latest stable
देखो)
            implementation("com.squareup.okhttp3:okhttp:<APP_JAISA_VERSION>")
            implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:<APP_JAISA>")
            implementation("org.jetbrains.kotlinx:kotlinx-coroutines-play-services:<APP_JAISA>")
            implementation("androidx.startup:startup-runtime:1.2.0")
            implementation("com.google.android.play:integrity:1.4.0")       // latest stable देखो
            testImplementation("junit:junit:4.13.2")
        }
```

`security/security.properties` (सिर्फ़ public जानकारी, secret नहीं)

```
gateway.url=https://secure.yourinstitute.in
tls.pins=sha256/PRIMARY_PIN_BASE64=,sha256/BACKUP_PIN_BASE64=
play.cloudProject=123456789012
pqc.kid=k-2026-11
pqc.mlkem.pub=<base64 of ML-KEM-768 public key, KeyTool से>
pqc.x25519.pub=<base64 of X25519 public key, KeyTool से>
```

`AndroidManifest.xml` (module का, merge होकर app में अपने-आप जुड़ता है)

```
        <manifest xmlns:android="http://schemas.android.com/apk/res/android"
                  xmlns:tools="http://schemas.android.com/tools">
            <application>
```

```
            <provider
                android:name="androidx.startup.InitializationProvider"
                android:authorities="${applicationId}.androidx-startup"
                android:exported="false"
                tools:node="merge">
                <meta-data android:name="com.institute.security.SecurityInitializer"
                           android:value="androidx.startup" />
            </provider>
        </application>
    </manifest>
```

`SecurityInitializer.kt` + `SecurityKit.kt`

```
    class SecurityInitializer : Initializer<SecurityKit> {
        override fun create(context: Context): SecurityKit {
            SecurityKit.init(context.applicationContext, SecurityConfig(
                gatewayBaseUrl = BuildConfig.GATEWAY_URL,
                tlsPins = BuildConfig.TLS_PINS.split(","),
                cloudProjectNumber = BuildConfig.PLAY_CLOUD_PROJECT.toLong(),
                pqcKid = BuildConfig.PQC_KID,
                pqcMlKemPub = Base64.decode(BuildConfig.PQC_MLKEM_PUB, Base64.NO_WRAP),
                pqcX25519Pub = Base64.decode(BuildConfig.PQC_X25519_PUB, Base64.NO_WRAP)))
            return SecurityKit
        }
        override fun dependencies(): List<Class<out Initializer<*>>> = emptyList()
    }

    data class SecurityConfig(
        val gatewayBaseUrl: String, val tlsPins: List<String>, val cloudProjectNumber: Long,
        val pqcKid: String, val pqcMlKemPub: ByteArray, val pqcX25519Pub: ByteArray)

    object SecurityKit {
        lateinit var appContext: Context; private set
        lateinit var config: SecurityConfig; private set
        /** app की तरफ़ से एक लाइन: "मेरा मौजूदा login token दो" */
        @Volatile var identityProvider: (suspend () -> String)? = null

        internal fun init(ctx: Context, cfg: SecurityConfig) { appContext = ctx; config = cfg }
        val http by lazy { SecureHttp.build(config) }
        val authz by lazy { AuthzClient(config, http) }
        val integrity by lazy { IntegrityProvider(appContext, config.cloudProjectNumber) }
    }
```

`device/DeviceKeyManager.kt` : Keystore में device-bound key

```
    object DeviceKeyManager {
        private const val ALIAS = "inst_device_key_v1"
        private val ks = KeyStore.getInstance("AndroidKeyStore").apply { load(null) }

        /** key न हो तो बनाओ. challenge = server की दी nonce (attestation के लिए). */
        fun ensureKey(attestationChallenge: ByteArray): Boolean {
            if (ks.containsAlias(ALIAS)) return true
            fun spec(strongBox: Boolean) = KeyGenParameterSpec.Builder(ALIAS, KeyProperties.PURPOSE_SIGN)
                .setAlgorithmParameterSpec(ECGenParameterSpec("secp256r1"))
                .setDigests(KeyProperties.DIGEST_SHA256)
                .setAttestationChallenge(attestationChallenge)
                .apply { if (strongBox && Build.VERSION.SDK_INT >= 28) setIsStrongBoxBacked(true) }
                .build()
            val kpg = KeyPairGenerator.getInstance(KeyProperties.KEY_ALGORITHM_EC, "AndroidKeyStore")
            return try { kpg.initialize(spec(true)); kpg.generateKeyPair(); true }
            catch (e: Exception) { kpg.initialize(spec(false)); kpg.generateKeyPair(); true }   //
StrongBox नहीं तो TEE
        }

        fun attestationChain(): List<String> =
            ks.getCertificateChain(ALIAS).map { Base64.encodeToString(it.encoded, Base64.NO_WRAP) }

        fun publicKey(): ECPublicKey = ks.getCertificate(ALIAS).publicKey as ECPublicKey
        fun sign(data: ByteArray): ByteArray = Signature.getInstance("SHA256withECDSA").run {
            initSign(ks.getKey(ALIAS, null) as PrivateKey); update(data); sign()      // DER format
```

```
        }
    }
```

`device/DpopSigner.kt` : हर request के लिए proof (RFC 9449)

```
    object DpopSigner {
        private fun b64u(b: ByteArray) = Base64.encodeToString(b, Base64.URL_SAFE or Base64.NO_PADDING or
Base64.NO_WRAP)
        private fun fix32(b: ByteArray): ByteArray { val t = b.dropWhile { it == 0.toByte()
}.toByteArray(); return ByteArray(32 - t.size) + t }

        private fun jwk(): org.json.JSONObject {
            val w = DeviceKeyManager.publicKey().w
            return org.json.JSONObject().put("kty", "EC").put("crv", "P-256")
                .put("x", b64u(fix32(w.affineX.toByteArray()))).put("y",
b64u(fix32(w.affineY.toByteArray())))
        }

        /** Keystore signature DER -> JWS raw R||S (64 bytes) */
        private fun derToRaw(der: ByteArray): ByteArray {
            var o = 2; if (der[1].toInt() and 0x80 != 0) o += der[1].toInt() and 0x7f
            val rl = der[o + 1].toInt() and 0xff; val r = der.copyOfRange(o + 2, o + 2 + rl); o += 2 + rl
            val sl = der[o + 1].toInt() and 0xff; val s = der.copyOfRange(o + 2, o + 2 + sl)
            return fix32(r) + fix32(s)
        }

        fun proof(method: String, url: String, accessToken: String?): String {
            val header = org.json.JSONObject().put("typ", "dpop+jwt").put("alg", "ES256").put("jwk",
jwk())
            val payload = org.json.JSONObject().put("htm", method).put("htu", url)
                .put("iat", System.currentTimeMillis() / 1000).put("jti",
java.util.UUID.randomUUID().toString())
            accessToken?.let {   // ath = base64url(SHA-256(access token))
                payload.put("ath", b64u(MessageDigest.getInstance("SHA-256").digest(it.toByteArray())))
            }
            val signingInput = b64u(header.toString().toByteArray()) + "." +
b64u(payload.toString().toByteArray())
            return signingInput + "." + b64u(derToRaw(DeviceKeyManager.sign(signingInput.toByteArray())))
        }
    }
```

`device/IntegrityProvider.kt` : Play Integrity (Standard API)

```
    class IntegrityProvider(private val ctx: Context, private val cloudProject: Long) {
        private var provider: StandardIntegrityManager.StandardIntegrityTokenProvider? = null

        private suspend fun prepare() {
            if (provider != null) return
            provider = IntegrityManagerFactory.createStandard(ctx).prepareIntegrityToken(
                StandardIntegrityManager.PrepareIntegrityTokenRequest.builder()
                    .setCloudProjectNumber(cloudProject).build()).await()
        }
        /** requestHash = SHA-256 of "action|objectId|timestamp|nonce". verdict इसी action से बँध जाता है. */
        suspend fun token(requestHash: String): String {
            prepare()
            return provider!!.request(StandardIntegrityManager.StandardIntegrityTokenRequest.builder()
                .setRequestHash(requestHash).build()).await().token()
        }
    }
```

`net/SecureHttp.kt` : pinning + हर request पर DPoP

```
    object SecureHttp {
        fun build(cfg: SecurityConfig): OkHttpClient {
            val host = java.net.URI(cfg.gatewayBaseUrl).host
            val pinner = CertificatePinner.Builder().apply { cfg.tlsPins.forEach { add(host, it) } }
}.build()
            return OkHttpClient.Builder().certificatePinner(pinner)
```

```
                    .addInterceptor(DpopInterceptor()).callTimeout(20, TimeUnit.SECONDS).build()
            }
        }

        class DpopInterceptor : Interceptor {
            override fun intercept(chain: Interceptor.Chain): Response {
                val orig = chain.request()
                val path = orig.url.encodedPath
                if (path.startsWith("/v1/session/") || path.startsWith("/v1/device/")) {   // token लेने वाले
calls
                    return chain.proceed(orig.newBuilder()
                        .header("DPoP", DpopSigner.proof(orig.method,
orig.url.toString().substringBefore("?"), null)).build())
                }
                val token = runBlocking { SessionManager.accessToken() }   // gateway का 10-मिनट token
                val url = orig.url.newBuilder().query(null).fragment(null).build().toString()
                return chain.proceed(orig.newBuilder()
                    .header("Authorization", "DPoP $token")
                    .header("DPoP", DpopSigner.proof(orig.method, url, token)).build())
            }
        }
```

`net/SessionManager.kt` **(ढाँचा):** (1) पहली बार: `GET /v1/device/challenge` → nonce; `DeviceKeyManager.ensureKey(nonce)`; `POST /v1/device/enroll` को chain + public JWK भेजो। (2) `POST /v1/session/exchange` में आपका *मौजूदा* login token (`SecurityKit.identityProvider`) भेजो; gateway उसे आपके मौजूदा auth provider की JWKS से verify करके `cnf.jkt` वाला 10-मिनट access token देता है। (3) expiry से 30 सेकंड पहले अपने-आप refresh।

> **Pinning की सावधानी:** हमेशा कम से कम एक **backup pin** रखो, वरना certificate बदलते ही app बंद हो जाएगा। पुरानी `network_security_config.xml` को हमने छुआ नहीं, ये pinning सिर्फ़ हमारे client की है।

6. Layer 3: AWS Nitro Enclave (Confidential Computing, Data-in-Use)

6.1 क्या मिलेगा

Enclave एक **अलग isolated VM** है (CPU/RAM parent EC2 से कटी हुई): **कोई network नहीं, कोई SSH नहीं, कोई persistent disk नहीं**; parent instance का root user भी अंदर नहीं झाँक सकता। बाहर से बात सिर्फ़ **vsock** से। असली ताक़त है **cryptographic attestation**: enclave अपनी image का hash (PCR0) AWS-signed document में KMS को दिखाता है, और KMS की key policy कहती है "Decrypt तभी जब PCR0 ये हो।" यानी private keys सिर्फ़ *आपकी approved image* के अंदर खुलती हैं।

> **ख़र्च/जटिलता का सच:** इसके लिए Nitro-supported EC2 instance (T-series नहीं) 24x7 चलाना पड़ता है, यानी अलग मासिक ख़र्च। छोटे institute के लिए ये **Phase 4 (आख़िरी)** रखो, पहले Layers 1, 2, 4।

6.2 Parent EC2 तैयार करना

```
        # Launch (Mumbai) : enclave-options Enabled=true ज़रूरी
        aws ec2 run-instances --region ap-south-1 --image-id <AL2023_AMI> \
          --instance-type m6i.xlarge --enclave-options 'Enabled=true' \
          --iam-instance-profile Name=institute-enclave-parent-profile \
          --key-name <key> --subnet-id <private-subnet> --security-group-ids <sg-private-only>

        # Instance पर (infra/parent-bootstrap.sh)
        sudo dnf install -y aws-nitro-enclaves-cli aws-nitro-enclaves-cli-devel docker socat
```

```
sudo usermod -aG ne,docker $USER
sudo systemctl enable --now docker
```

`infra/allocator.yaml` → `/etc/nitro_enclaves/allocator.yaml` में:

```
memory_mib: 2048     # enclave के लिए आरक्षित RAM (JVM के लिए >=1536)
cpu_count: 2
```

`infra/vsock-proxy.yaml` → `/etc/nitro_enclaves/vsock-proxy.yaml` (enclave को सिर्फ़ KMS तक पहुँचने दो):

```
allowlist:
- {address: kms.ap-south-1.amazonaws.com, port: 443}
```

```
sudo systemctl enable --now nitro-enclaves-allocator.service
# KMS तक vsock proxy (enclave -> parent -> KMS)
vsock-proxy 8000 kms.ap-south-1.amazonaws.com 443 --config /etc/nitro_enclaves/vsock-proxy.yaml &
# Gateway -> Enclave bridge (localhost:5005  ->  enclave CID 16 port 5005)
socat TCP-LISTEN:5005,bind=127.0.0.1,fork VSOCK-CONNECT:16:5005 &
```

6.3 Enclave image: `enclave/Dockerfile`

```
FROM eclipse-temurin:21-jre-jammy
RUN apt-get update && apt-get install -y --no-install-recommends socat && rm -rf /var/lib/apt/lists/*
# kmstool + libnsm: github.com/aws/aws-nitro-enclaves-sdk-c के README से build करके vendor/ में रखो
COPY vendor/kmstool_enclave_cli /usr/local/bin/kmstool_enclave_cli
COPY vendor/libnsm.so /usr/lib/libnsm.so
COPY build/libs/enclave-all.jar /app/enclave.jar
COPY run.sh /app/run.sh
RUN chmod +x /app/run.sh /usr/local/bin/kmstool_enclave_cli
CMD ["/app/run.sh"]
```

`enclave/run.sh`:

```
#!/bin/bash
# vsock (parent से आने वाला traffic) -> enclave के अंदर के localhost:8080 HTTP server
socat VSOCK-LISTEN:5005,fork TCP:127.0.0.1:8080 &
exec java -Xmx1200m -jar /app/enclave.jar
```

6.4 Enclave app: `EnclaveMain.kt` (मुख्य हिस्से)

```
object EnclaveMain {
    @Volatile private var keys: HybridKem.RecipientPrivate? = null
    private const val THRESHOLD = 0.80f      // अपने face model के हिसाब से tune करो

    @JvmStatic fun main(a: Array<String>) {
        val s = HttpServer.create(InetSocketAddress("127.0.0.1", 8080), 0)

        // Boot पर parent एक बार भेजता है: temp AWS creds + KMS-wrapped keys blob
        s.createContext("/init") { ex ->
            val j = JSONObject(ex.requestBody.readBytes().decodeToString())
            val out = ProcessBuilder("kmstool_enclave_cli", "decrypt",
                "--region", "ap-south-1", "--proxy-port", "8000",
                "--aws-access-key-id", j.getString("ak"),
                "--aws-secret-access-key", j.getString("sk"),
                "--aws-session-token", j.getString("st"),
                "--ciphertext", j.getString("wrapped")).redirectErrorStream(true)
                .start().inputStream.readBytes().decodeToString()
            val raw = Base64.getDecoder().decode(out.substringAfter("PLAINTEXT:").trim())
            keys = HybridKem.RecipientPrivate(j.getString("kid"), raw.copyOfRange(0, 2400),
raw.copyOfRange(2400, 2432))
            ex.sendResponseHeaders(204, -1)
        }
```

```
                // Purpose-specific API: plaintext कभी बाहर नहीं जाता, सिर्फ़ नतीजा
                s.createContext("/face/verify") { ex ->
                    val k = keys ?: run { ex.sendResponseHeaders(503, -1); return@createContext }
                    val j = JSONObject(ex.requestBody.readBytes().decodeToString())
                    val aad = j.getString("aad").toByteArray()
                    val probe = floats(HybridKem.open(Sealed.fromJson(j.getJSONObject("probe")), aad, k))
                    val tmpl  = floats(HybridKem.open(Sealed.fromJson(j.getJSONObject("template")),
"tmpl".toByteArray(), k))
                    val score = cosine(probe, tmpl)
                    val body = JSONObject().put("match", score >= THRESHOLD).put("score",
score).toString().toByteArray()
                    ex.sendResponseHeaders(200, body.size.toLong()); ex.responseBody.use { it.write(body) }
                }
                s.start()
            }
            private fun floats(b: ByteArray) = java.nio.ByteBuffer.wrap(b).asFloatBuffer().let { f ->
FloatArray(f.remaining()).also(f::get) }
            private fun cosine(a: FloatArray, b: FloatArray): Float {
                var d = 0f; var na = 0f; var nb = 0f
                for (i in a.indices) { d += a[i] * b[i]; na += a[i] * a[i]; nb += b[i] * b[i] }
                return d / (kotlin.math.sqrt(na) * kotlin.math.sqrt(nb) + 1e-9f)
            }
        }
```

(Face matching सिर्फ़ उदाहरण है, ये तब लागू है जब आपके app में matching server पर होती हो। अगर आपकी matching on-device है तो यही pattern fee/PII processing के लिए use करो।)

6.5 Build, run, PCR0 नोट करना

```
        cd enclave && ./gradlew shadowJar && docker build -t institute-enclave:1 .
        nitro-cli build-enclave --docker-uri institute-enclave:1 --output-file institute.eif
        #   Output में मिलेगा:  "PCR0": "<hex>"  <-- यही KMS policy में जाएगा
        nitro-cli run-enclave --eif-path institute.eif --cpu-count 2 --memory 2048 --enclave-cid 16
        nitro-cli describe-enclaves
        # ध्यान: --debug-mode में PCRs शून्य (000..) आते हैं; production में debug-mode कभी नहीं
```

6.6 KMS key + Policy (`infra/kms-key-policy.json` का मुख्य statement)

```
        {
          "Sid": "AllowDecryptOnlyFromApprovedEnclave",
          "Effect": "Allow",
          "Principal": {"AWS": "arn:aws:iam::<ACCOUNT_ID>:role/institute-enclave-parent-role"},
          "Action": "kms:Decrypt",
          "Resource": "*",
          "Condition": {
            "StringEqualsIgnoreCase": {"kms:RecipientAttestation:ImageSha384": "<PCR0_HEX_FROM_BUILD>"}
          }
        }
```

Key administrators को `kms:Decrypt` **मत** दो (सिर्फ़ Encrypt/Manage)। इससे AWS का कोई admin भी बिना approved enclave के private keys नहीं खोल पाएगा। Image बदली (नया deploy) → नया PCR0 → policy में **manual approval** के साथ अपडेट (Section 8 का workflow इसे enforce करता है)।

6.7 Boot sequence (parent से enclave को init)

Parent पर एक छोटा script: IMDS से instance-role की temp credentials लेकर, wrapped-keys blob (S3/SSM Parameter Store से) के साथ `POST http://127.0.0.1:5005/init` करता है। इसके बाद keys सिर्फ़ enclave की RAM में रहती हैं; enclave restart = दोबारा init।

7. Layer 4: Post-Quantum Cryptography (PQC)

7.1 क्यों और कहाँ

- **ख़तरा:** attacker आज encrypted traffic/डेटा **जमा** कर लेता है ("harvest now, decrypt later") और भविष्य के quantum computer से RSA/ECC तोड़ता है। Biometric templates और student/fee records जैसा *लंबी उम्र वाला* डेटा सबसे ज़्यादा असुरक्षित है।
- **हल:** NIST का **ML-KEM-768** (FIPS 203) key-encapsulation, **हाइब्रिड** रूप में X25519 के साथ: एक टूटे तब भी दूसरा बचाए। Payload **AES-256-GCM** से।
- **कहाँ:** यहाँ हम *application-layer envelope* बनाते हैं: संवेदनशील payload app में seal होता है और सिर्फ़ enclave (या gateway) खोल सकता है। ये TLS से **अलग और अतिरिक्त** है।
- **TLS के बारे में सच:** Chrome/WebView और कई CDN/load balancers hybrid PQ key exchange (X25519MLKEM768) देने लगे हैं, पर native Android का OkHttp/Conscrypt stack उसे negotiate करे ये आपके Android version/library पर निर्भर है। इसीलिए envelope ज़रूरी है। सर्वर TLS पर hybrid policy जहाँ उपलब्ध हो, चालू करो। Certificates की signatures अभी classical ही हैं।
- **Signatures (वैकल्पिक):** ML-DSA (FIPS 204) से policy bundles / audit-log checkpoints sign कर सकते हो। ये Phase 5 की बात है।

7.2 `shared-crypto/.../HybridKem.kt` (app + gateway + enclave तीनों में एक ही कोड)

```
        package com.institute.crypto

        import org.bouncycastle.crypto.agreement.X25519Agreement
        import org.bouncycastle.crypto.digests.SHA256Digest
        import org.bouncycastle.crypto.generators.HKDFBytesGenerator
        import org.bouncycastle.crypto.params.*
        import org.bouncycastle.pqc.crypto.mlkem.*
        import java.security.*
        import javax.crypto.Cipher
        import javax.crypto.spec.*

        object HybridKem {
            private const val INFO = "institute-hybrid-kem-v1"
            private val rng = SecureRandom()

            class RecipientPublic(val kid: String, val mlkem: ByteArray, val x25519: ByteArray)
            class RecipientPrivate(val kid: String, val mlkem: ByteArray, val x25519: ByteArray)
            class Sealed(val kid: String, val kemCt: ByteArray, val ephX25519: ByteArray, val nonce:
ByteArray, val ct: ByteArray)

            /** एक बार (KeyTool से): सर्वर/enclave की long-term चाबियाँ */
            fun generateRecipient(kid: String): Pair<RecipientPublic, RecipientPrivate> {
                val g = MLKEMKeyPairGenerator().apply { init(MLKEMKeyGenerationParameters(rng,
MLKEMParameters.ml_kem_768)) }
                val kp = g.generateKeyPair()
                val xPriv = X25519PrivateKeyParameters(rng)
                return Pair(
                    RecipientPublic(kid, (kp.public as MLKEMPublicKeyParameters).encoded,
xPriv.generatePublicKey().encoded),
                    RecipientPrivate(kid, (kp.private as MLKEMPrivateKeyParameters).encoded, xPriv.encoded))
            }

            /** App की तरफ़: seal. aad = संदर्भ ("attendance|batch|ts|nonce") ताकि replay/मिलावट न हो */
            fun seal(plain: ByteArray, aad: ByteArray, to: RecipientPublic): Sealed {
                val enc = MLKEMGenerator(rng).generateEncapsulatedSecret(
                    MLKEMPublicKeyParameters(MLKEMParameters.ml_kem_768, to.mlkem))
                val eph = X25519PrivateKeyParameters(rng)
                val ephPub = eph.generatePublicKey().encoded
                val ssX = ByteArray(32)
                X25519Agreement().apply { init(eph) }.calculateAgreement(X25519PublicKeyParameters(to.x25519,
0), ssX, 0)
                val key = derive(enc.secret, ssX, to.kid, enc.encapsulation, ephPub)
```

```
                val nonce = ByteArray(12).also(rng::nextBytes)
                return Sealed(to.kid, enc.encapsulation, ephPub, nonce, gcm(Cipher.ENCRYPT_MODE, key, nonce,
aad, plain))
            }

            /** Enclave की तरफ़: open */
            fun open(s: Sealed, aad: ByteArray, me: RecipientPrivate): ByteArray {
                val ssKem = MLKEMExtractor(MLKEMPrivateKeyParameters(MLKEMParameters.ml_kem_768,
me.mlkem)).extractSecret(s.kemCt)
                val ssX = ByteArray(32)
                X25519Agreement().apply { init(X25519PrivateKeyParameters(me.x25519, 0)) }
                    .calculateAgreement(X25519PublicKeyParameters(s.ephX25519, 0), ssX, 0)
                val key = derive(ssKem, ssX, s.kid, s.kemCt, s.ephX25519)
                return gcm(Cipher.DECRYPT_MODE, key, s.nonce, aad, s.ct)
            }

            // दोनों secrets + पूरा transcript (ciphertexts) HKDF में: एक भी टूटे तो key सुरक्षित
            private fun derive(ssKem: ByteArray, ssX: ByteArray, kid: String, kemCt: ByteArray, ephPub:
ByteArray): ByteArray {
                val salt = MessageDigest.getInstance("SHA-256").digest(kemCt + ephPub)
                val h = HKDFBytesGenerator(SHA256Digest()).apply { init(HKDFParameters(ssKem + ssX, salt,
"$INFO|$kid".toByteArray())) }
                return ByteArray(32).also { h.generateBytes(it, 0, 32) }
            }
            private fun gcm(mode: Int, key: ByteArray, nonce: ByteArray, aad: ByteArray, data: ByteArray):
ByteArray =
                Cipher.getInstance("AES/GCM/NoPadding").run {
                    init(mode, SecretKeySpec(key, "AES"), GCMParameterSpec(128, nonce)); updateAAD(aad);
doFinal(data)
                }
        }
```

> BouncyCastle के class/constructor नाम version के साथ थोड़े बदल सकते हैं (ML-KEM lightweight API 1.79+ में है)। अगर compile error आए तो अपने version के docs से मिलाओ। नीचे का unit test पक्का करता है कि कोड सही चल रहा है।

Unit test (`security/src/test/.../HybridKemTest.kt`)

```
        class HybridKemTest {
            @Test fun roundTrip() {
                val (pub, priv) = HybridKem.generateRecipient("k1")
                val aad = "attendance|b1|1700000000|n1".toByteArray()
                val s = HybridKem.seal("hello".toByteArray(), aad, pub)
                assertEquals("hello", String(HybridKem.open(s, aad, priv)))
            }
            @Test fun wrongAadFails() {
                val (pub, priv) = HybridKem.generateRecipient("k1")
                val s = HybridKem.seal("x".toByteArray(), "a".toByteArray(), pub)
                assertFailsWith<Exception> { HybridKem.open(s, "b".toByteArray(), priv) }
            }
        }
```

7.3 चाबियाँ बनाना और KMS से लपेटना (एक बार, admin के लैपटॉप पर)

```
        # gateway/tools/KeyTool.kt चलाओ: keys.bin = mlkem_priv(2400B) || x25519_priv(32B) बनाता है,
        # और public हिस्से base64 में print करता है -> security.properties में डालो
        ./gradlew :tools:run        # (या  kotlinc -script  की तरह)

        aws kms encrypt --region ap-south-1 --key-id alias/institute-enclave \
          --plaintext fileb://keys.bin --query CiphertextBlob --output text > wrapped.b64
        shred -u keys.bin           # plaintext चाबियाँ तुरंत नष्ट; सिर्फ़ wrapped.b64 रखो (S3/SSM में)
```

Encrypt करने के लिए attestation नहीं चाहिए; **Decrypt** सिर्फ़ approved enclave कर सकता है (Section 6.6)। Private keys कभी Git में, logs में या किसी के लैपटॉप पर नहीं रहनी चाहिए।

7.4 Key rotation

- हर ~90 दिन में नया `kid` (जैसे `k-2027-02`) generate करो; app config में नया public key डालकर release करो।
- Gateway/enclave **पिछले 2 kid** accept करते हैं (पुराने app versions के लिए), फिर पुराना हटा दो।
- App में minimum-version check रखो ताकि बहुत पुराने app बंद किए जा सकें।

7.5 App में उपयोग (`pqc/SecureEnvelope.kt`)

```
object SecureEnvelope {
    fun sealFor(action: String, objectId: String, plain: ByteArray): org.json.JSONObject {
        val cfg = SecurityKit.config
        val nonce = java.util.UUID.randomUUID().toString()
        val aad = "$action|$objectId|${System.currentTimeMillis() / 1000}|$nonce"
        val s = HybridKem.seal(plain, aad.toByteArray(),
            HybridKem.RecipientPublic(cfg.pqcKid, cfg.pqcMlKemPub, cfg.pqcX25519Pub))
        fun b(x: ByteArray) = Base64.encodeToString(x, Base64.NO_WRAP)
        return org.json.JSONObject().put("aad", aad).put("sealed", org.json.JSONObject()
            .put("kid", s.kid).put("kemCt", b(s.kemCt)).put("eph", b(s.ephX25519))
            .put("nonce", b(s.nonce)).put("ct", b(s.ct)))
    }
}
```

Gateway `replay_nonces` table से aad का nonce/timestamp जाँचता है (10 मिनट window, एक nonce एक ही बार), ताकि पकड़ा हुआ payload दोबारा न चल सके।

8. GitHub Actions: सिर्फ़ नई workflow फ़ाइलें

पुराने workflows को हाथ नहीं लगाया। सब `sec-` prefix वाली नई फ़ाइलें हैं और अपने-अपने `paths:` पर ही चलती हैं (यानी पुराने काम पर कोई असर नहीं)। Action versions के लिए latest stable major देख लेना।

`.github/workflows/sec-authz-model-test.yml`

```
name: sec-authz-model-test
on:
  pull_request: {paths: ['authz/**']}
  push: {branches: [main], paths: ['authz/**']}
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: OpenFGA model tests
        run: docker run --rm -v "$PWD/authz:/w" -w /w openfga/cli model test --tests model.fga.yaml
```

`.github/workflows/sec-gateway-ci.yml`

```
name: sec-gateway-ci
on:
  pull_request: {paths: ['security-gateway/**', 'shared-crypto/**', 'enclave/**']}
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: {distribution: temurin, java-version: '21'}
      - uses: gradle/actions/setup-gradle@v4
      - run: ./gradlew test
        working-directory: security-gateway
```

`.github/workflows/sec-android-module.yml` : नया module बिगड़े नहीं इसकी जाँच

```
        name: sec-android-module
        on:
          pull_request: {paths: ['security/**', 'shared-crypto/**']}
        jobs:
          build:
            runs-on: ubuntu-latest
            steps:
              - uses: actions/checkout@v4
              - uses: actions/setup-java@v4
                with: {distribution: temurin, java-version: '17'}
              - uses: gradle/actions/setup-gradle@v4
              - run: ./gradlew :security:testDebugUnitTest :security:lintDebug
```

`.github/workflows/sec-enclave-build.yml` : Image build + PCR0 निकालना + **manual approval**

```
        name: sec-enclave-build
        on:
          workflow_dispatch:
          push: {branches: [main], paths: ['enclave/**', 'shared-crypto/**']}
        permissions: {id-token: write, contents: read}     # AWS OIDC: कोई long-lived AWS key नहीं
        jobs:
          build:
            runs-on: ubuntu-latest
            environment: production-enclave                # Settings > Environments: required reviewers लगाओ
            steps:
              - uses: actions/checkout@v4
              - uses: actions/setup-java@v4
                with: {distribution: temurin, java-version: '21'}
              - uses: aws-actions/configure-aws-credentials@v4
                with:
                  role-to-assume: arn:aws:iam::<ACCOUNT_ID>:role/gh-enclave-build
                  aws-region: ap-south-1
              - run: ./gradlew shadowJar
                working-directory: enclave
              - id: ecr
                uses: aws-actions/amazon-ecr-login@v2
              - name: Build + push image
                run: |
                  IMG=${{ steps.ecr.outputs.registry }}/institute-enclave:${{ github.sha }}
                  docker build -t "$IMG" enclave && docker push "$IMG"
                  echo "IMG=$IMG" >> $GITHUB_ENV
              - name: Build EIF on the Nitro parent (via SSM) and read PCR0
                run: |
                  CMD=$(aws ssm send-command --instance-ids ${{ vars.ENCLAVE_PARENT_ID }} \
                    --document-name AWS-RunShellScript \
                    --parameters "commands=['/opt/institute/build-eif.sh $IMG']" \
                    --query Command.CommandId --output text)
                  aws ssm wait command-executed --command-id "$CMD" --instance-id ${{ vars.ENCLAVE_PARENT_ID
}}
                  aws ssm get-command-invocation --command-id "$CMD" --instance-id ${{ vars.ENCLAVE_PARENT_ID
}} \
                    --query StandardOutputContent --output text | tee build.txt
              - uses: actions/upload-artifact@v4
                with: {name: enclave-measurements, path: build.txt}
              # PCR0 को KMS policy में डालना जान-बूझकर MANUAL review के बाद: एक reviewer build.txt देखकर
              # infra/kms-key-policy.json में नया PCR0 अपडेट करता है।
```

`build-eif.sh` (parent पर): image pull → `nitro-cli build-enclave ...` → measurements print।

9. Rollout Plan और Testing

Phase	क्या करो	"Done" की शर्त
0 (≈1 हफ़्ता)	OpenFGA + `model.fga` + tests + tuples migration script (पुरानी roles से tuples बनाओ)	CI में सब tests pass; `fga query check` वही देता है जो आपकी पुरानी rules देती हैं
1	Gateway + `security_db` + device enrollment + DPoP, **shadow mode** (`ENFORCE=false`)	1-2 हफ़्ते के audit log में झूठे `WOULD_DENY` ≈ 0
2	`ENFORCE=true` , पहले fee और attendance routes पर ; admin routes पर STRONG tier	Real devices (सस्ते फ़ोन सहित) पर test; support के लिए "device reset" प्रक्रिया तैयार
3	PQC envelope: face template और fee payload seal करना	Unit tests + पुराना डेटा नए format में migrate; key rotation की रिहर्सल
4	Nitro Enclave: keys enclave में, KMS policy PCR0 से बँधी	Attestation से ही `Decrypt` चले, बिना उसके `AccessDenied` आए

Testing checklist

- Teacher A, Teacher B के batch की attendance नहीं लगा पाए (403)।
- Student अपने ही fee record देखे ; दूसरे का नहीं ; Parent सिर्फ़ अपने बच्चे का।
- Rooted/emulator फ़ोन से admin portal: reject।
- Access token किसी दूसरे डिवाइस/टूल (curl) में copy करके चलाओ : **fail** (DPoP key नहीं है)।
- पुराना payload replay करो : reject (nonce)। Payload के `aad` में 1 अक्षर बदलो : decrypt fail।
- Gateway/OpenFGA बंद करो : सब कुछ **deny** हो, allow नहीं (fail-closed)।
- Enclave का debug-mode चलाओ : KMS `Decrypt` fail होना चाहिए (PCR शून्य)।
- खोया फ़ोन : admin `revoked_at` set करे → उसी क्षण उस डिवाइस की सारी requests reject।

सामान्य ग़लतियाँ

1. सारी permissions सिर्फ़ app के अंदर check करना (bypass हो जाती है)।
2. OpenFGA/Enclave को public internet पर खोलना।
3. TLS pin में backup pin न रखना।
4. हर action पर STRONG integrity माँगना (सस्ते फ़ोन बाहर)।
5. Private keys को Git/CI secrets में रखना, उन्हें सिर्फ़ KMS-wrapped रखो।
6. Debug-mode enclave को production में चलाना।

10. आगे पढ़ने के लिए (Official references)

- Zanzibar paper: research.google/pubs/zanzibar-googles-consistent-global-authorization-system
- OpenFGA docs: openfga.dev/docs
- BeyondCorp: research.google/pubs/beyondcorp-a-new-approach-to-enterprise-security
- DPoP (RFC 9449): rfc-editor.org/rfc/rfc9449
- Play Integrity API: developer.android.com/google/play/integrity
- Android Keystore key attestation: developer.android.com/privacy-and-security/security-key-attestation
- AWS Nitro Enclaves: docs.aws.amazon.com/enclaves/latest/user/nitro-enclave.html

- KMS + Nitro Enclaves attestation condition keys: docs.aws.amazon.com/kms/latest/developerguide/conditions-nitro-enclaves.html
- aws-nitro-enclaves-sdk-c (kmstool): github.com/aws/aws-nitro-enclaves-sdk-c
- NIST FIPS 203 (ML-KEM): csrc.nist.gov/pubs/fips/203/final
- BouncyCastle: bouncycastle.org

यह गाइड एक starter blueprint है। Production से पहले security review और penetration test ज़रूर कराएँ।